

Working with EBVcube Data: A Practical Workflow Guide for BES-SIM 2

Andrés Mármol-Guijarro, Emmanuel Ocegüera

2026-03-16

Contents

0.1	Introduction	2
0.2	Required Packages	2
1	1. Downloading Cube Files from the NextCloud Repository	2
2	2. Setting Up Relative Paths	2
3	3. Accessing the Data Cubes	5
4	4. Accessing Layers Within the Cube	7
4.1	4.1 Plotting layers within a EBVcube	8
5	5. Subsetting High-Resolution Cubes (10 arc-seconds) by Region	9
5.1	5.1 Bounding Box — EU Extent	9
5.2	5.2 Shapefile Subset — EU Administrative Boundary	10
6	6. Saving GeoTIFF Files	11
6.1	6.1 From a loaded SpatRaster	11
6.2	6.2 Direct write during read (preferred for large subsets)	12
6.3	6.3 Via <code>terra::writeRaster()</code> for format control	12
7	Quick Reference	12
7.1	7.1 Further Resources	13

0.1 Introduction

This guide walks you through the core workflow for accessing **EBVcube netCDF files** using the **ebvcube** R package. EBVcube is a standardised format for **Essential Biodiversity Variable (EBV)** data — gridded, multi-dimensional datasets tracking biodiversity change across space and time. A single **.nc** file can contain multiple data cubes organised hierarchically by scenario and metric, each with four dimensions: longitude, latitude, time, and entity (e.g., a species, land-use class, or ecological group). The datasets used throughout this guide are the **EBV cubes produced under BES-SIM 2**. For more information, visit the **ebvcube** repository.

0.2 Required Packages

Restarting and running on a clean environment

```
rm(list = ls())
```

```
library(ebvcube)
library(terra)
library(geodata)
```

If BioConductor dependencies (**rhdf5**, **DelayedArray**, **HDF5Array**) are missing, install them via **BiocManager** before loading **ebvcube**.

```
#install.packages(c("ebvcube", "terra"))

#install.packages("BiocManager")
#BiocManager::install(c("rhdf5", "DelayedArray", "HDF5Array"))
```

1. Downloading Cube Files from the NextCloud Repository

The BES-SIM 2 EBVcube **.nc** files are hosted on a shared NextCloud folder. Two folders have been created: **Data_geo** for downloading all input datasets and **BES_results** to upload results, reports and figures. Links and passwords have been circulated with the modelling teams. In case you have lost your password, please contact: a.marmol@idiv.de.

2. Setting Up Relative Paths

The notebook is designed to work with a mirrored folder structure. Anchoring the working directory to the script's location means all paths resolve correctly on any machine replicating the same layout — no absolute paths needed.

Input data

```

Data_geo\
  climate_data\
    ISIMIP3b\
    WorldClim\
  land_use\
    GLOBIO\
      cubes\          # GLOBIO BES-SIM 2 EBVcube .nc files
      raw\
      code\
    PLUM\
      cubes\          # PLUM BES-SIM 2 EBVcube .nc files
      raw\
      code\
  protected_areas\
    GLOBIO-based_PAs\
    PLUM-based_PAs\

```

Outputs

```

BES_results\
  EBVcube_ensemble_Tutorial\  # outputs from this notebook
  results_biodiversity\
    Model1\
      EBVcubes\
      figures\
      reports\
    Model2\
      EBVcubes\
      figures\
      reports\
  results_ecosystems\
    Model3\
      EBVcubes\
      figures\
      reports\
    Model4\
      EBVcubes\
      figures\
      reports\

```

Setting relative paths to read input files

```

# Anchor to script location (RStudio only)
setwd(dirname(rstudioapi::getActiveDocumentContext())$path))
# Alternative for non-RStudio environments:
# setwd(getSrcDirectory(function() {})[1])

# EBVcube inputs
root_globio <- "../Data_geo/land_use/GLOBIO/cubes/"
root_plum   <- "../Data_geo/land_use/PLUM/cubes/"

# Protected areas
root_globiopa <- "../Data_geo/protected_areas/GLOBIO-based_PAs/"
root_plumpa  <- "../Data_geo/protected_areas/PLUM-based_PAs/"

```

Setting relative paths for outputs (can be deleted if not relevant for this tutorial)

```
# Outputs - tutorial
root_tutorial <- "../..//BES_results/EBVcube_ensemble_Tutorial/"

# Outputs - model folders (template: repeat for each model)
# Replace "results_biodiversity" / "results_ecosystems" and "Model1" as needed
root_m1 <- "../..//BES_results/results_biodiversity/Model1/"
root_m1_cubes <- file.path(root_m1, "EBVcubes/")
root_m1_figures <- file.path(root_m1, "figures/")
root_m1_reports <- file.path(root_m1, "reports/")
```

Once the paths are set, it is now possible to list all the land use cubes for GLOBIO or PLUM.

```
nc_globio <- list.files(root_globio, pattern = "\\\\.nc$", full.names = TRUE, ignore.case = TRUE)
nc_plum <- list.files(root_plum, pattern = "\\\\.nc$", full.names = TRUE, ignore.case = TRUE)

nc_plumpa <- list.files(root_plumpa, pattern = "\\\\.nc$", full.names = TRUE, recursive = TRUE, ignore.case = TRUE)
nc_globiopa <- list.files(root_globiopa, pattern = "\\\\.nc$", full.names = TRUE, recursive = TRUE, ignore.case = TRUE)

cat("GLOBIO cubes:\n"); print(nc_globio)
```

GLOBIO cubes:

```
## [1] "../..//Data_geo/land_use/GLOBIO/cubes//alkemade_ecostr_id103_20260216.nc"
## [2] "../..//Data_geo/land_use/GLOBIO/cubes//alkemade_ecostr_id98_20260223.nc"
```

```
cat("PLUM cubes:\n"); print(nc_plum)
```

PLUM cubes:

```
## [1] "../..//Data_geo/land_use/PLUM/cubes//alexander_croscut_id101_20260202.nc"
## [2] "../..//Data_geo/land_use/PLUM/cubes//alexander_ecostr_id100_20260204.nc"
## [3] "../..//Data_geo/land_use/PLUM/cubes//alexander_ecostr_id102_20260204.nc"
## [4] "../..//Data_geo/land_use/PLUM/cubes//alexander_ecostr_id102_20260309.nc"
## [5] "../..//Data_geo/land_use/PLUM/cubes//alexander_ecostr_id99_20260217.nc"
```

```
cat("GLOBIO protected areas cubes:\n"); print(nc_globiopa)
```

GLOBIO protected areas cubes:

```
## [1] "../..//Data_geo/protected_areas/GLOBIO-based_PAs//cubes/alkemade_ecostr_idTBD_20260309.nc"
```

```
cat("PLUM protected areas cubes:\n"); print(nc_plumpa)
```

PLUM protected areas cubes:

```
## [1] "../..//Data_geo/protected_areas/PLUM-based_PAs//cubes/alexander_ecostr_idTBD_20260306.nc"
```

Files within each vector are referenced by index throughout (`nc_globio[1]`, `nc_plum[2]`, etc.). Adjust indices to match your local file inventory. The following table provides an index to all the input data cubes, which are indexed in the code box below.

Filepath	Data cube name	Short name
<code>alkemade_ecostr_id103_20260216.nc</code>	Upscaled 0.5 Land Use projections (BES-SIM 2 GLOBIO)	<code>globio_lu_05</code>
<code>alkemade_ecostr_id98_20260223.nc"</code>	Land Use projection (BES-SIM 2 GLOBIO)	<code>globio_lu</code>
<code>alexander_ecostr_id99_20260217.nc"</code>	Land Use projection (BES-SIM 2 LandSymm-PLUM)	<code>plum_lu</code>
<code>alexander_ecostr_id100_20260204.nc"</code>	Forest management (BES-SIM 2 LandSyMM-PLUM)	<code>plum_frstmng</code>
<code>alexander_croscut_id101_20260202.nc"</code>	Crop areas and intensities (BES-SIM 2 LandSyMM-PLUM)	<code>plum_crop_int</code>
<code>alexander_ecostr_id102_20260309.nc"</code>	Native 0.5 Land Use projection (BES-SIM 2 LandSymm-PLUM)	<code>plum_lu_05</code>
<code>alkemade_ecostr_idTBD_20260309.nc</code>	GLOBIO Protected area layers	<code>globio_pas</code>
<code>alexander_ecostr_idTBD_20260306.nc</code>	LandSyMM-PLUM Protected area layers	<code>plum_pas</code>

```
dc <- list(
  globio_lu_05 = "../../../Data_geo/land_use/GLOBIO/cubes//alkemade_ecostr_id103_20260216.nc",
  globio_lu    = "../../../Data_geo/land_use/GLOBIO/cubes//alkemade_ecostr_id98_20260223.nc",
  plum_lu      = "../../../Data_geo/land_use/PLUM/cubes//alexander_ecostr_id99_20260217.nc",
  plum_frstmng = "../../../Data_geo/land_use/PLUM/cubes//alexander_ecostr_id100_20260204.nc",
  plum_crop_int = "../../../Data_geo/land_use/PLUM/cubes//alexander_croscut_id101_20260202.nc",
  plum_lu_05   = "../../../Data_geo/land_use/PLUM/cubes//alexander_ecostr_id102_20260309.nc",
  globio_pas   = "../../../Data_geo/protected_areas/GLOBIO-based_PAs//cubes/alkemade_ecostr_idTBD_20260309.nc",
  plum_pas     = "../../../Data_geo/protected_areas/PLUM-based_PAs//cubes/alexander_ecostr_idTBD_20260306.nc"
)
```

Then call any cube using `dc$data_cube_name` (e.g. `dc$plum_lu`)

3. Accessing the Data Cubes

`ebv_properties()` returns an S4 object with all file-level metadata. The key slots for exploration are `general`, `spatial`, and `temporal`.

Start by loading the properties of one file and inspecting the available slots:

```
globio05_dc <- ebv_properties(dc$globio_lu_05[1], verbose = FALSE)
slotNames(globio05_dc)
```

```
## [1] "general" "spatial" "temporal" "metric" "scenario" "ebv_cube"
```

The `general` slot contains descriptive metadata — the title, a description of the dataset, and the list of entity names. Entity names are important because their index position maps directly to the `entity` argument in `ebv_read*` functions:

```
globio05_dc@general$title
```

```
## [1] "Upscaled Land Use projections (BES-SIM 2 GLOBIO)"
```

```
globio05_dc@general$description
```

```
## [1] "Land use projections built from the three Sustainable Development Pathways and reinterpreted to
```

```
globio05_dc@general$entity_names    # entity index maps directly to the entity argument in ebv_read*
```

```
## [1] "Urban"                "Cropland"
## [3] "Potential secondary vegetation" "Agroforestry"
## [5] "Pasture"              "Rangeland"
## [7] "Forestry"             "Secondary vegetation"
## [9] "Unmanaged forest"     "Shrubland/Grassland"
## [11] "Other"                "Bare Areas"
```

The `spatial` slot reports the coordinate reference system, cell resolution, and bounding extent of the grid:

```
cat("EPSG:      ", globio05_dc@spatial$epsg,      "\n")
```

```
## EPSG:      4326
```

```
cat("Resolution:", globio05_dc@spatial$resolution, "\n")
```

```
## Resolution: 0.5 0.5
```

```
cat("Extent:    ", globio05_dc@spatial$extent,    "\n")
```

```
## Extent:     -180 180 -90 90
```

The `temporal` slot lists all available time steps as dates. This tells you the temporal coverage of the cube and helps you identify which `timestep` index to pass when reading a layer:

```
cat(as.character(globio05_dc@temporal$dates), sep = "\n")
```

```
## 2020-01-01
## 2030-01-01
## 2050-01-01
## 2070-01-01
```

Note: Not all EBV cubes have time steps. The protected area cubes for GLOBIO and LandSyMM-PLUM, for instance, contain only one mask layer per scenario with no associated year. This is an example of the latter.

```
# getting LandSyMM-PLUM properties
plum_pas <- ebv_properties(dc$plum_pas[1], verbose = FALSE)

# checking is temporal metadata
cat(as.character(plum_pas@temporal$dates), sep = "\n")
```

```
## 2000-01-01
```

The time step within it shows 2000-01-01, which has been set as default as a placeholder for the temporal slot by the ebvcube R-package.

4. Accessing Layers Within the Cube

The internal hierarchy is **Scenario** → **Metric** → **Entity** (scenarios are optional). `ebv_datacubepaths()` resolves this and returns a data frame of all valid paths, with descriptions of the metric and scenario each path represents.

```
globio05_paths <- ebv_datacubepaths(dc$globio_lu_05, verbose = FALSE)
print(globio05_paths)
```

```
##           datacubepaths      scenario_names metric_names
## 1 scenario_1/metric_1/ebv_cube Nature for Nature   Land cover
## 2 scenario_2/metric_1/ebv_cube Nature as Culture   Land cover
## 3 scenario_3/metric_1/ebv_cube Nature for Society   Land cover
## 4 scenario_4/metric_1/ebv_cube      SSP2-Baseline   Land cover
```

In this file there are four scenarios — *Nature for Nature*, *Nature as Culture*, *Nature for Society*, and *SSP2-Baseline* — each containing the same metric (Land cover). The `datacubepaths` column gives the internal HDF5 path used to locate a specific cube within the file. This path is passed to subsequent functions to specify which cube to work with.

In the example, `datacubepath = globio05_paths[1, 1]` selects the first scenario — “**Nature for Nature**” — which is then used to retrieve the cube’s properties via `ebv_properties()`. Alternatively, “**Nature as Culture**” could be selected using `datacubepath = globio05_paths[2, 1]`.

The `@metric` and `@ebv_cube$type` slots provide additional metadata about the selected datacube, describing the metric (“Land cover”) and the data type used to store the values, respectively.

```
dc_path <- globio05_paths[1, 1]
globio05_dc <- ebv_properties(dc$globio_lu_05[1], dc_path, verbose = FALSE)

# description of the metric
globio05_dc@metric

## $name
## [1] "Land cover"
##
## $description
```

```
## [1] "Land cover records whether a given land cover type occurs within the grid (e.g. percentage of land cover type)"
##
## $units
## [1] "percentage of the grid"
```

```
# description of the data type (i.e. float, interger, boolean)
globio05_dc@ebv_cube$type
```

```
## [1] "H5T_IEEE_F32LE"
```

4.1 4.1 Plotting layers within a EBVcube

`ebv_read()` loads a specific entity and timestep as a `SpatRaster` (`type = "r"`) or array (`type = "a"`). At this point it is advisable to recall all the entities within each cube using `globio@general$entity_names`.

```
globio05_dc@general$entity_names
```

```
## [1] "Urban" "Cropland"
## [3] "Potential secondary vegetation" "Agroforestry"
## [5] "Pasture" "Rangeland"
## [7] "Forestry" "Secondary vegetation"
## [9] "Unmanaged forest" "Shrubland/Grassland"
## [11] "Other" "Bare Areas"
```

The output indicates the numbers with their corresponding names within `globio_lu_05` ebvcube. With this information it is now simple to choose a desired entity to plot. In this example, `entity = 9` (“Unmanaged forest”) and `timestep = 1` (corresponding to the first available time step in the dataset that in this case is “2020”) have been selected. This cube will display projections made for “Nature for Nature” scenarios as specified by `datacube_path = dc_path` (see above). These parameters could be changed to query other entities and timesteps accordingly.

```
start_time <- Sys.time()

data_layer <- ebv_read(
  filepath      = dc$globio_lu_05,
  datacube_path = dc_path,           # this is selecting Nature for Nature projections.
  entity        = 5,
  timestep       = 1,
  ignore_RAM     = TRUE
)

cat("Read time:", round(Sys.time() - start_time, 2), "seconds\n")
plot(data_layer)
```

Alternatively, all the available time steps can be plotted using `timestep = 1:4`.

```
start_time <- Sys.time()

data_layer <- ebv_read(
  filepath      = dc$globio_lu_05[1],
  datacube_path = dc_path,
```



```

entity      = 5,
timestep    = 1:4,
ignore_RAM  = TRUE
)

cat("Read time:", round(Sys.time() - start_time, 2), "seconds\n")
plot(data_layer)

```

In this case, `lyr.1`, `lyr.2`, `lyr.3` and `lyr.4` correspond to 2020, 2030, 2050 and 2070, respectively.

NOTE: For global 10 arc-second files, always use `ignore_RAM = TRUE` together with a spatial subset (Section 5) — loading the full extent is not feasible on standard workstations.

5. Subsetting High-Resolution Cubes (10 arc-seconds) by Region

At 10 arc-second resolution, a global layer contains approximately 1.3 billion cells, which can exceed the available RAM. The package provides two spatial subsetting functions that extract only the requested region directly from the file on disk, avoiding the need to load the full global extent into memory.

Function	Input	Notes
<code>ebv_read_bb()</code>	Bounding box <code>c(xmin, xmax, ymin, ymax)</code>	Fast rectangular clip
<code>ebv_read_shp()</code>	Path to a <code>.shp</code> file	Clips to boundary, masks exterior cells to NA

5.1 Bounding Box — EU Extent

For the following exercise, we now switch to read `dc$globio_lu`. The bounding box below covers mainland EU in EPSG 4326.

For this is necessary to redefine the datacube paths by reading `dc$globio_lu`

```

globio_paths <- ebv_datacube_paths(dc$globio_lu, verbose = FALSE)
print(globio_paths)

##                datacube_paths    scenario_names metric_names
## 1 scenario_1/metric_1/ebv_cube Nature for Nature   Land cover
## 2 scenario_2/metric_1/ebv_cube Nature as Culture   Land cover
## 3 scenario_3/metric_1/ebv_cube Nature for Society   Land cover
## 4 scenario_4/metric_1/ebv_cube      SSP2 RCP 2.6    Land cover

```

Selecting “Nature for Society” by using `globio_paths[3, 1]` to plot.

```

dc_path <- globio_paths[3, 1]
globio_dc <- ebv_properties(dc$globio_lu, dc_path, verbose = FALSE)

```

Revealing entities within the cube.

```
globio_dc@general$entity_names
```

```
## [1] "Urban" "Cropland"
## [3] "Potential secondary vegetation" "Agroforestry"
## [5] "Pasture" "Rangeland"
## [7] "Forestry" "Secondary vegetation"
## [9] "Unmanaged forest" "Shrubland/Grassland"
## [11] "Other" "Bare Areas"
```

entity = 8 “Forestry” for all time steps is plotted

```
br_bb <- c(-74, -28, -34, 6) # xmin, xmax, ymin, ymax

br_subset <- ebv_read_bb(
  filepath      = dc$globio_lu,
  datacube_path = dc_path,
  entity        = 8,
  timestep      = 1:4,
  bb            = br_bb,
  ignore_RAM    = TRUE,
  verbose       = TRUE
)

plot(br_subset)
```

5.2 Shapefile Subset — EU Administrative Boundary

Subsetting the dataset using shapefiles, for instance of countries, is also possible. For this example, the dataset will be subset to be plotted within Brazilian boundaries. Shapefiles are downloaded using `geodata` package.

```
#calling geodata package
library(geodata)

#setting tempdir within environment
temp_dir = '../Temp'
dir.create(temp_dir, showWarnings = FALSE, recursive = TRUE)

# Download Brazil level 0 boundary
brazil_shp <- geodata::gadm(country = "BRA", level = 0, path = temp_dir)

# Write to shapefile in temp_dir (required by ebv_read_shp)
brazil_shp_path <- file.path(temp_dir, "brazil.shp")
terra::writeVector(brazil_shp, brazil_shp_path, overwrite = TRUE)
```

Once downloaded, Brazil shapefile is then indicated in `shp` parameter of the `ebv_read_shp()` function.

```
globio_br <- ebv_read_shp(
  filepath      = dc$globio_lu,
```

```

datacubepath = dc_path,
entity       = 8,                               # Forestry
timestep     = 1:4,
shp          = brazil_shp_path,
ignore_RAM   = TRUE,
verbose      = TRUE
)

plot(globio_br)

```

6. Saving GeoTIFF Files

6.1 From a loaded SpatRaster

The `ebv_write()` function exports a subset or processed EBV object to disk as a GeoTIFF file. It accepts the in-memory data object and a specified output path. The resulting tiff will be written in the folder that was recently created for saving Brazil shapefile.

Note: the type of data selected to save the dataset matters for the resulting size of the final tif. Selection will depend on the metric that needs to be written as output. For instance, `globio_lu` has only binary land cover type, meaning that 16-bit INT2S would work. FLT8S (64-bit) is recommended for biodiversity indices and modelled outputs requiring maximum numerical precision.

The table below summarizes the available GeoTIFF data types supported by the `type` argument:

Code	Type	Bits	Range	Use Case
INT1S	Signed integer	8-bit	-128 to 127	Compact categorical data
INT2S	Signed integer	16-bit	-32,768 to 32,767	Elevation, temperature anomalies
INT2U	Unsigned integer	16-bit	0 to 65,535	Land cover classes, counts
INT4S	Signed integer	32-bit	-2.1B to 2.1B	Large integer values
INT4U	Unsigned integer	32-bit	0 to 4.3B	Large positive integer values
FLT4S	Signed float	32-bit	$\sim \pm 3.4 \times 10^{38}$	Continuous data, moderate precision
FLT8S	Signed float	64-bit	$\sim \pm 1.8 \times 10^{308}$	High precision continuous data

```

ebv_write(
  data      = br_subset,
  outputpath = file.path(temp_dir, "br_entity9_t1.tif"), # fixed filename
  overwrite  = TRUE,
  verbose    = TRUE,
  type      = "INT2S"
)

```

6.2 6.2 Direct write during read (preferred for large subsets)

Passing `outputpath` to `ebv_read_bb()` or `ebv_read_shp()` writes directly to disk and returns the path rather than a `SpatRaster`, avoiding loading into RAM. However, the downside of writing directly through the `ebv_read_shp()` is that writing data type cannot be parametrised, which may render larger tif files.

```
ebv_read_shp(  
  filepath      = dc$globio_lu,  
  datacubeopath = globio_paths[1,1], #this is Nature for Nature scenario  
  entity        = 9,  
  timestep      = 1:4,  
  shp           = brazil_shp_path,  
  outputpath    = file.path(temp_dir, "br_entity9_t1.tif"),  
  overwrite     = TRUE,  
  ignore_RAM    = TRUE  
)
```

6.3 6.3 Via `terra::writeRaster()` for format control

Use this when specific datatype, compression, or tiling is required. Match `datatype` to `globio_dc@ebv_cube$type` to avoid precision loss or unnecessary file size.

```
terra::writeRaster(  
  x            = br_subset,  
  filename     = file.path(temp_dir, "br_entity9_t1.tif"),  
  filetype     = "GTiff",  
  datatype     = "INT2S", # or INT2S / INT4U depending on the cube's data type  
  overwrite    = TRUE,  
  gdal         = c("COMPRESS=DEFLATE")  
)
```

7 Quick Reference

Step	Function	Notes
File metadata	<code>ebv_properties(filepath)</code>	Returns S4 object; key slots: general , spatial , temporal
List cubes	<code>ebv_datacube_paths(filepath)</code>	One row per scenario/metric combination
Cube metadata	<code>ebv_properties(filepath, datacubeopath)</code>	Metric description and data type
Read full layer	<code>ebv_read()</code>	Use only for coarser resolutions
Bounding box subset	<code>ebv_read_bb()</code>	Essential for 10 arc-second data
Shapefile subset	<code>ebv_read_shp()</code>	Masks exterior cells to NA
Save (ebvcube)	<code>ebv_write()</code>	Straightforward write from <code>SpatRaster</code>

Step	Function	Notes
Save (terra)	<code>terra::writeRaster()</code>	Full format and compression control

7.1 Further Resources

- [ebvcube GitHub](#) — source code and issue tracker
- [ebvcube CRAN](#) — stable release and function reference
- [EBV Data Portal](#) — GEO BON dataset catalogue
- [EBVCube format specification](#)
- [terra documentation](#)